variable. This is called a compilation environment macro,[4] and it returns the current module name as an atom.

It's common to have a new function in the same module that defines the struct, which takes arguments to build the struct. Another common pattern is modifier functions that take the struct as the first argument and return a modified version.

This technique allows structs and classes to feel similar to each other, even though they are different things. But you'll still need to enumerate over your data structures to read or modify them. Let's look at some techniques for doing that.

Enumerating Data Structures

You'll perform two types of operations on your data. Iteration reads the contents of the data structure, so you can do something meaningful with it. Map (or reduce) transforms the contents of the data structure to modify it or to create something totally different.

Elixir has several options for iterating and mapping your data structures. We're going to look at a few of the built-in options that Elixir gives for iteration and mapping. We'll cover the Enum module and comprehensions.

Enum Module

The Enum[5] module is arguably the most useful module in the standard library. It does lots of things, but we'll cover some of the most commonly used functions. It's built on top of the Enumerable protocol, which lets us enumerate maps and lists out of the box.

Enumerable Protocol



Protocols allow code to change behavior based on the input data. Enum relies on the Enumerable protocol to support all of the functions in the Enum module.

You can implement a protocol for any data type or struct. This means that your own data structure could implement Enumerable and you could use Enum functions with it.

The use case varies for protocols, but it's not a beginner-level feature. So we won't cover protocols in this book.

4. https://hexdocs.pm/elixir/Macro.html#expand_once/2
5. https://hexdocs.pm/elixir/1.14/Enum.html